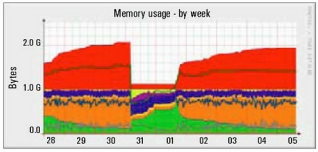


Figure 1: Munin memory graph

resource like the disk, memory, etc, on your network using Munin. You can view the health of your servers at *http://munin.sastratechnologies.net*

Figure 1 shows a graph of the memory usage of one the servers. The red area indicates that the server is swapping heavily. This server is badly in need of more memory and needs to be resized.

***Devel()***

This Drupal module provides valuable information to Drupal module developers. It logs query times and highlights slow queries. It also provides the memory consumption information when it initialises and at shutdown.

***XHPRof***

Originally developed by Facebook, XHPRof is a function level hierarchical profiler for PHP. This PECL package can be integrated with your Drupal installation to report CPU times and memory usage of your modules.

***Command line***

Some knowledge of the command line is essential to analyse the consumption of hardware resources. Though Munin plots a neat graph, the command line is essential to get the numbers.

Let's get started now. First identify the bottlenecks using Pingdom (you could use Site24x7 or Yslow, if you wish) and identify which of the components need to be fixed. Then either remove the bottleneck or adjust the parameters to tune the component.

## Check your site using Pingdom

Measuring performance and tweaking your stack is iterative and you'll usually end up doing three to five round trips. Start the exercise by checking the site using Pingdom. Visit *http://tools.pingdom.com* and provide your site's URL. Within a few minutes, you'll have a neat bar graph of all the requests done to load your URL, a performance grade score, the number of requests, the load time and the page size. In our case, mileage varied from site to site. In one case the browser waited for 5.95 seconds for the home page to load, and the total time to load the site was 11.76 seconds! In another case, the browser took 657 milliseconds to download *menu.css* which was about 2.4 KB. In both these cases, the root causes were more or less similar.

- There were query strings in the URL for a few images
- Two requests, one for a CSS file and another an image, were returning 404/410 responses
- JavaScript assets were not being cached

At this point you should understand that not all of these symptoms were present in a single site—these are two different sites. However, for the rest of the article, I will refer to a single site and demonstrate how each task has improved the performance. At the point at which the site's performance is acceptable, I will illustrate further tuning exercises by picking up some of my other sites.

## HTTP compression

Enabling compression on Apache decreases the response payload size. Present day browsers accept compressed content and requesting for it saves bandwidth. To enable delivery of such content, enable the *mod_deflate* module.

To check whether your site is actually returning content that is compressed, you can view the response headers. To do so, use the command line and type:

```
wget -S www.yourdomain.in
```

After you make these changes to Apache, test the site using Pingdom to see if there is any improvement. In our case, Pingdom returned a response time of 7.93 seconds, a significant improvement from the 11.76 seconds.

## Aggregate CSS and JS assets

Yahoo recommends aggregation of CSS and JS. In Drupal, you can do this in the *Home | Administration | Configuration | Development* page. Check the boxes in the Bandwidth Optimization box. Pingdom reported 6.11 seconds after we enabled aggregation.

## Configure a file-based caching system called Boost

Boost is a contributed Drupal module that provides static page caching. If your site receives mostly anonymous traffic you should install Boost. Before you install it, you should configure Drupal to output clean URLs for Boost to work. Clean URLs in Drupal can be enabled only if Apache has the *Mod_Rewrite* module enabled.

Boost generates *.htaccess* rules that you should copy and paste in your *.htaccess* folder. Remember that if you have followed the instructions in previous sections and set *AllowOverride None* you should enable *Override* for the folder in which your *.htaccess* resides. Boost adds its signature to each page that Drupal generates. You can view the signature by looking at the source of the page and at the very bottom you will see the following line:

```
<!-- Page cached by Boost @ 2013-03-30 05:51:41, expires @
2013-04-06 05:51:41, lifetime 1 week -->
```